DOCUMENTATION DE LA DÉMARCHE

Amélioration IA du contrôle qualité des données - Projet Bottleneck

Veille, cahier des charges, hypothèses, tests, résultats, limites et décisions

Rémy Daura · Projet 13 - Pilotez un projet data augmenté par l'IA · OpenClassrooms

Sommaire

1. Synthèse
2. Veille métier et technologique
3. Cahier des charges fonctionnel
4. Mini-plan de formation pour les métiers
5. Hypothèses et tests réalisés
6. Résultats
7. Décisions - ce qui est gardé, écarté, et pourquoi
8. Limites et pistes d'amélioration
9. Reproductibilité
10. Annexes

1. Synthèse

Ce document consolide la démarche suivie pour la Mission 1 du Projet 13 : améliorer, grâce à l'IA, de façon critique et documentée, le livrable du Projet 6 (données du caviste Bottleneck).

Le besoin de départ était simple : le contrôle qualité des données reposait sur une dizaine de vérifications manuelles éparpillées dans un notebook, sans garantie de reproductibilité sur de futurs exports. Trois méthodes de fiabilisation ont été comparées, non pas sur documentation seule mais par test réel sur les données : deux librairies de validation (Pandera, Great Expectations) et une méthode sans code, consistant à transmettre les données brutes à une IA générative.

Résultat central : la méthode IA a retrouvé la quasi-totalité des anomalies déjà connues, mais a surtout révélé deux angles morts dans mon propre schéma de validation et deux catégories d'anomalies entièrement nouvelles. Cette découverte a changé la décision finale, initialement pensée comme un choix binaire entre deux outils : la démarche retenue combine désormais une exploration ponctuelle par IA générative et une formalisation reproductible dans un schéma Pandera.

Au passage, cette démarche a mis au jour deux bugs de reproductibilité réels dans le notebook P6 original, qui ne s'exécutait pas entièrement sans intervention avant cette mission.

2. Veille métier et technologique

2.1 Besoin de veille

Le notebook P6 identifie manuellement 9 catégories d'anomalies dans les données Bottleneck (prix négatifs, stocks incohérents, doublons, orphelins, ventes à perte…), réparties dans une dizaine de cellules distinctes. Cette approche fonctionne, mais rien ne garantit qu'elle sera rejouée à l'identique sur les prochains exports mensuels, et rien n'alerte automatiquement en cas de nouvelle anomalie.

Besoin formulé : identifier une ou plusieurs méthodes de validation de données qui centralisent ces règles de qualité, de façon reproductible et automatisable, adaptées à une petite structure sans équipe data engineering dédiée.

2.2 Solutions comparées

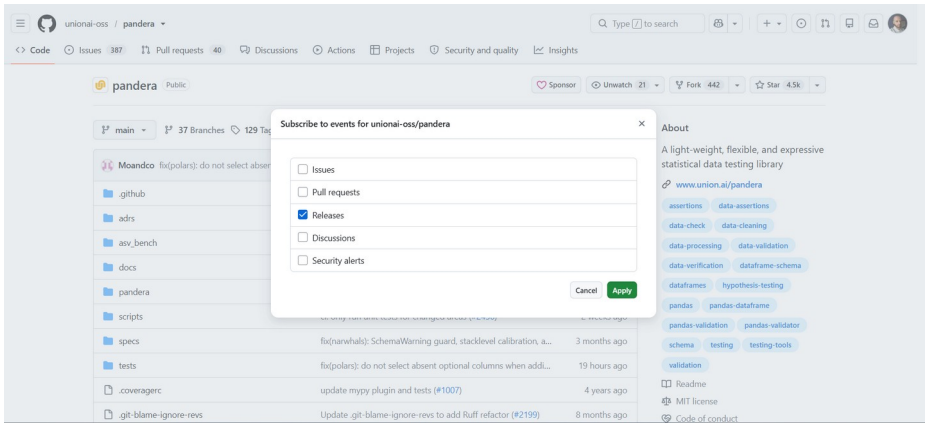
Trois approches ont été comparées, d'abord sur documentation sourcée puis par test empirique réel sur les données Bottleneck.

Critère	Pandera	Great Expectations	IA générative directe
Mise en œuvre (mesuré)	~9 lignes pour 6 règles	~15 lignes pour les mêmes règles (+65%)	Aucun code, une consigne en langage naturel
Reproductibilité	Code versionné, rejouable en < 1s	Idem, plus lourd à versionner ; Data Docs HTML auto-générées (vérifié fonctionnel)	Non reproductible automatiquement d'une exécution à l'autre
Sobriété numérique	12 dépendances	107 dépendances	Aucune installation, coût de calcul externalisé
Couverture observée sur ce projet	5/5 anomalies de prix/stock	5/5, identique à Pandera	Quasi-totalité des anomalies connues + 2 catégories inédites

Sources : Endjin (mars 2026), Medium/Bhagya Rana (oct. 2025), Aeturrell (mars 2025), Techoc Blog (janv. 2026), documentation officielle Pandera - liens complets en annexe.

2.3 Automatisation de la veille

Pour rester informé des évolutions de Pandera sans démarche active répétée, un abonnement aux notifications GitHub Releases a été activé sur le dépôt officiel unionai-oss/pandera (Watch → Custom → Releases). Capture ci-dessous : la case Releases cochée et le bouton "Unwatch" confirment un abonnement actif.



2.4 Décision et justification

Outil retenu pour l'usage courant : Pandera - meilleur rapport simplicité/couverture, confirmé par test empirique, et mieux adapté à un notebook porté par une seule personne, sans pipeline de production multi-outils. Great Expectations est écarté de l'usage courant, à réévaluer si Bottleneck multiplie ses sources de données ou ses équipes.

3. Cahier des charges fonctionnel

3.1 Contexte et parties prenantes

Bottleneck est un caviste dont les outils de gestion de données sont actuellement artisanaux : trois exports distincts (ERP, site Web, table de liaison) doivent être rapprochés manuellement pour produire une vision fiable du catalogue et des ventes.

Partie prenante	Rôle dans le projet
Nicolas	Responsable des ventes, commanditaire initial de l'analyse (P6), destinataire des rapports
CODIR	Utilisateur final des analyses ; décide des actions correctives sur les anomalies identifiées
Équipe IT	Gère l'ERP et le site Web, source potentielle de corrections en amont
Stagiaire	Maintient la table de liaison entre les identifiants ERP et Web
Rémy Daura	Analyste, responsable de la fiabilisation du contrôle qualité des données

Le CODIR s'appuie sur le catalogue nettoyé et les analyses associées (chiffre d'affaires, marge, stock) pour arbitrer sur trois anomalies critiques déjà identifiées : rentabilité, intégration des données, gestion de stock. Ce même livrable sert également de point de départ à un futur projet de data visualisation évoqué par Nicolas.

3.2 Problématique reformulée

Le contrôle qualité des données de catalogue repose aujourd'hui sur des vérifications manuelles ponctuelles, exécutées une fois dans un notebook, sans garantie qu'elles soient rejouées à l'identique sur les prochains exports mensuels.

Comment fiabiliser durablement ce contrôle qualité, de façon reproductible et automatisable, sans complexifier l'outillage au-delà des besoins réels d'une petite structure sans équipe data engineering dédiée ?

3.3 État actuel

- Notebook Python de 99 cellules, exécuté manuellement à chaque nouvel export
- 9 catégories d'anomalies déjà identifiées manuellement, réparties dans une dizaine de cellules distinctes
- Aucun garde-fou automatique en cas de nouvelle anomalie sur un futur export
- Aucune trace centralisée des règles de qualité attendues sur les données
- Catalogue final : 714 références actives sur 825 références ERP d'origine

3.4 Périmètre

Dans le périmètre :

- Fiabilisation de la validation des 3 fichiers sources actuels : erp.csv, web.csv, liaison.csv
- Détection automatisée des 9 catégories d'anomalies déjà documentées lors du P6
- Documentation de la démarche comparative (veille, décisions, tests, limites)

Hors périmètre :

- Automatisation complète d'un pipeline de production (ordonnancement, CI/CD)
- Remplacement de l'ERP ou du CMS du site Web
- Gestion des stocks en temps réel
- Extension à d'autres sources de données non mentionnées par Bottleneck

Période et population : données d'extraction au 31 octobre ; ventes du 1er au 31 octobre. Catalogue de vin Bottleneck : 825 références ERP, 714 références actives après nettoyage.

3.5 Contraintes

Type de contrainte	Détail
Données manquantes	85 lignes du fichier Web sans code SKU ; valeurs nulles diverses déjà recensées
Qualité des identifiants	product_id (ERP) est numérique, sku (Web) est parfois non numérique - hétérogénéité déjà source d'erreurs
RGPD	Exports composés de données produit (catalogue), aucune donnée personnelle identifiée à ce stade
Outillage	Python déjà en place ; aucun budget identifié pour un outil payant ou une solution cloud
Ressources humaines	Une seule personne côté data chez Bottleneck, pas d'équipe data engineering dédiée
Délais	Mission réalisée dans le cadre du calendrier de fin de formation (Projet 13)

3.6 Objectifs, enjeux et priorités

Objectif principal : qu'une anomalie de données déjà connue soit détectée automatiquement dès l'import du fichier, sans dépendre d'une relecture manuelle exhaustive du notebook.

Enjeu : le futur projet de data visualisation évoqué par Nicolas dépendra directement de la fiabilité de cette base.

Éléments prioritaires :

- Détection automatique des 9 catégories d'anomalies déjà connues
- Non-régression : conserver un périmètre de données final au moins équivalent à la méthode manuelle (714 références actives)
- Documentation et traçabilité des règles de qualité et des décisions prises
- Sensibilisation de Nicolas et de l'équipe commerciale à la lecture des rapports de validation

3.7 Critères de réussite (KPI)

Dimension	Critère
Data	100% des 9 catégories d'anomalies historiques détectées automatiquement par le schéma de validation
Data	0% de valeurs manquantes tolérées sur les colonnes clés (product_id, sku, price) après validation
Modèle / Insights	Volume de références actives conservé à périmètre de règles égal (714 références)
Opérationnel	Temps d'exécution du contrôle qualité inférieur à 5 secondes
Opérationnel	Reproductibilité totale : même résultat obtenu à chaque exécution sur les mêmes données
Opérationnel	Clarté du rendu : chaque anomalie signalée précise la ligne concernée et la règle violée

3.8 Ressources et budget

Type	Détail	Coût
Humaines	Rémy Daura (analyste, seul contributeur technique)	-
Techniques	Python 3, pandas (déjà utilisés)	0 €
Techniques	Pandera (librairie open source)	0 €

3.9 Jalons, livrables et planning

Lot	Livrable	Statut
Lot 1 - Cadrage	Veille technologique + cahier des charges fonctionnel	Réalisé
Lot 2 - POC	Schémas de validation testés empiriquement (Pandera, Great Expectations, IA)	Réalisé
Lot 3 - Intégration	Section dédiée dans le notebook P6 amélioré	Réalisé
Lot 4 - Documentation	Ce document + gestion de projet	Réalisé
Lot 5 - Restitution	Portfolio professionnel + préparation de la soutenance	En cours

4. Mini-plan de formation pour les métiers

Public concerné : Nicolas et l'équipe commerciale, qui liront les résultats de validation sans écrire de code.

Objectif pédagogique	Format / durée	Accessibilité
Comprendre ce qu'est une "règle de validation"	Support 1 page, langage non technique - 15 min	Police ≥ 12pt, contraste élevé, jamais d'information portée par la seule couleur
Savoir lire le résumé produit par le script	Démonstration commentée sur un	Tableau structuré, compatible lecteur

Objectif pédagogique	Format / durée	Accessibilité
	export réel - 20 min	d'écran si diffusé en PDF/Word balisé
Savoir qui contacter en cas d'anomalie détectée	Fiche réflexe 1 page - 10 min	Alternative textuelle systématique à tout pictogramme

5. Hypothèses et tests réalisés

Hypothèse 1 - un schéma de validation suffit à automatiser le contrôle qualité

Test 1 (Pandera) : schéma construit sur les 3 fichiers bruts. Résultat : 5 anomalies de prix/stock détectées, avec les mêmes identifiants que ceux documentés manuellement lors du P6.

Test 2 (Great Expectations) : mêmes règles, testées sur les mêmes données brutes. Résultat : détection strictement identique, pour un coût de mise en œuvre plus élevé. La fonctionnalité "Data Docs" a été vérifiée par génération réelle.

Hypothèse 2 - une IA générative sans règle prédéfinie peut égaler ou dépasser ces méthodes structurées

Née à la demande du mentor de suivi. Protocole : dans une conversation neuve, les 3 fichiers bruts ont été transmis avec pour seule consigne d'identifier toute anomalie, sans indication préalable. Prompt exact et résultat brut conservés en annexe.

Résultat : la méthode a retrouvé les anomalies déjà connues (mêmes identifiants exacts), et a surtout révélé deux limites réelles dans mon propre schéma Pandera, plus deux catégories d'anomalies entièrement nouvelles (fiches en doublon, ventes négatives).

Test complémentaire - correction et re-validation

Les schémas Pandera et Great Expectations ont été corrigés (règle de stock rendue bidirectionnelle, calcul des orphelins décomposé en deux causes distinctes) puis re-exécutés pour confirmer que les résultats recoupent désormais exactement les chiffres historiques du P6.

6. Résultats

Ce que la méthode IA a mis en évidence	Impact
Un 2e sens de l'incohérence stock_status/quantité, non testé initialement	Règle corrigée dans le schéma Pandera et le notebook
Décomposition exacte des "orphelins" : 91 + 20, contre un chiffre agrégé de 111 initialement	Règle scindée en deux, cohérente avec le P6 d'origine
Risque de collision d'identifiants (SKU réutilisé entre deux produits)	Documenté ; non encore automatisé
6 fiches produit en doublon (même titre, SKU différents)	Nouvelle règle ajoutée aux schémas
2 compteurs de ventes négatifs	Nouvelle règle ajoutée aux schémas

Par ailleurs, l'intégration du POC dans le notebook a permis de découvrir et corriger deux bugs de reproductibilité réels : une colonne (stock_status_2) supprimée deux fois, et une variable de marge (taux_marge) calculée sur un tableau de données puis utilisée sur un autre qui ne la possédait pas. Le notebook original ne s'exécutait donc pas entièrement sans intervention avant cette mission.

7. Décisions - ce qui est gardé, écarté, et pourquoi

Élément	Décision	Pourquoi
Pandera pour la validation reproductible	Gardé	Meilleur rapport simplicité/couverture, confirmé par test empirique
Great Expectations en usage courant	Écarté	Couverture identique à Pandera, ~65% de code en plus
IA générative directe comme contrôle de production	Écartée	Non reproductible automatiquement ; utilisée en amont comme méthode d'exploration
Règle stock_status bidirectionnelle	Ajoutée	Trouvée manquante grâce au test de la méthode IA
Décomposition de la règle "orphelins"	Corrigée	Le chiffre agrégé initial ne recoupait pas le chiffre historique P6
Risque de collision d'identifiants	Documenté, non automatisé	Nécessite une règle plus complexe qu'une contrainte de colonne simple

8. Limites et pistes d'amélioration

- Le risque de collision d'identifiants trouvé par la méthode IA reste documenté mais pas encore traduit en règle automatisée.
- La méthode IA générative n'est pas reproductible automatiquement d'une exécution à l'autre : elle sert à explorer, pas à surveiller en continu.
- Le choix de Pandera serait à réévaluer si le volume de données ou le nombre d'équipes impliquées venait à grandir significativement.
- La gestion de projet (backlog détaillé, registre des risques) est formalisée dans un document séparé, livré avec ce dossier.

9. Reproductibilité

Environnement

Composant	Version utilisée
Python	3.12
pandas	≥ 2.0
pandera	0.32.1
great_expectations	1.21.0

Structure des fichiers

- erp.csv, web.csv, liaison.csv - données sources brutes
- poc_pandera_validation.py - script de validation Pandera (autonome)
- poc_great_expectations.py - script de validation Great Expectations (autonome)
- Daura_Remy_P6_ameliore_P13.ipynb - notebook P6 amélioré, section "Amélioration IA"

Instructions d'exécution

1. Installer les dépendances : pip install pandas pandera great_expectations openpyxl

2. Placer les 3 fichiers CSV sources dans le même dossier que les scripts.

3. Lancer : python poc_pandera_validation.py, puis python poc_great_expectations.py.

4. Pour le notebook complet : ouvrir Daura_Remy_P6_ameliore_P13.ipynb dans Jupyter et exécuter toutes les cellules dans l'ordre.

10. Annexes

- Annexe A - Script de validation Pandera (poc_pandera_validation.py)
- Annexe B - Script de validation Great Expectations (poc_great_expectations.py)
- Annexe C - Trace de l'essai IA directe : prompt exact et résultat brut (annexe_essai_ia_directe.md)
- Annexe D - Notebook P6 amélioré (Daura_Remy_P6_ameliore_P13.ipynb)
- Annexe E - Gestion de projet : backlog et registre des risques (gestion_projet.md, document séparé)